

1. Research and compare different configuration management tools (Ansible, Puppet, Chef) and write a report on their use cases and advantages?

Comparison Report: Configuration Management Tools (Ansible, Puppet, Chef)

Overview

Feature	Ansible	Puppet	Chef
Language	YAML (declarative)	Puppet DSL (declarative)	Ruby (DSL - imperative/declarative)
Agentless	Yes (via SSH)	Requires agent	Requires agent
Learning Curve	Easy	Moderate	Steep
OS Support	Linux, macOS, Windows	Linux, Windows	Linux, Windows
Use Case	Simple deployments, fast adoption	Complex environments, large-scale	Custom configurations, DevOps-heavy
Configuration Style	Push-based	Pull-based (master-agent)	Pull-based (master- agent)
Community/Support	Strong, Red Hat-backed	Strong, supported by Perforce	Strong, backed by Progress
Popular Use Cases	Web servers, CI/CD, cloud provisioning	Infrastructure compliance, policy mgmt	Complex deployments, hybrid infra

Use Cases & Advantages

Ansible

- **Best for:** Simplicity, smaller teams, and agentless environments.
- **Advantages:**
 - Agentless (via SSH) → easier to manage.
 - Simple syntax (YAML).
 - Easy integration with CI/CD tools (Jenkins, GitLab CI).
 - Ideal for provisioning, app deployment, and config enforcement.

Puppet

- **Best for:** Enterprise environments with strict compliance needs.
- **Advantages:**
 - Strong support for desired state configuration.
 - Extensive module ecosystem.
 - Good reporting and GUI (Puppet Enterprise).

Scalable and secure.

Chef

- **Best for:** Complex infrastructures requiring programmable logic.
- **Advantages:**
 - Full flexibility via Ruby.
 - Integrates with cloud platforms and containers.

- **Test-driven development and compliance tools (InSpec, Chef Automate).**
 - **Ideal for large-scale DevOps workflows.**
 -
-

Summary Recommendation

Scenario	Recommended Tool
Quick automation, CI/CD setup	Ansible
Policy-driven enterprise control	Puppet
Complex environments with custom logic	Chef

- 2. Extend the CI pipeline to include continuous delivery using Jenkins or GitLab CI. Implement a blue-green deployment strategy?**

CI/CD Pipeline with Blue-Green Deployment in Jenkins

Prerequisites:

- **Jenkins installed with:**
 - **Git plugin**
 - **Pipeline plugin**

- SSH plugin (optional)
 - Two environments: Blue (active), Green (staging)
-

Sample Jenkins Pipeline (Blue-Green Deployment with Nginx)

groovy

pipeline {

agent any

environment {

BLUE_SERVER = "192.168.1.10"

GREEN_SERVER = "192.168.1.11"

ACTIVE_COLOR = "blue" // Assume blue is live initially

}

stages {

stage('Clone Repo') {

steps {

git 'https://github.com/example/webapp.git'

}

}

```
stage('Build & Test') {
```

```
  steps {
```

```
    sh './build.sh'
```

```
    sh './test.sh'
```

```
  }
```

```
}
```

```
stage('Deploy to Green') {
```

```
  steps {
```

```
    sshagent(['deploy-key']) {
```

```
      sh "scp -r ./dist/* user@${GREEN_SERVER}:/var/www/html"
```

```
    }
```

```
  }
```

```
}
```

```
stage('Switch Traffic') {
```

```
  steps {
```

```
    sshagent(['deploy-key']) {
```

```
      sh "ssh user@loadbalancer 'update_lb green'"
```

```
    }
```

```
  }
```

```
}
```

```
stage('Health Check') {  
    steps {  
        sh "curl -f http://${GREEN_SERVER} || exit 1"  
    }  
}
```

```
stage('Deactivate Blue') {  
    steps {  
        sshagent(['deploy-key']) {  
            sh "ssh user@${BLUE_SERVER} 'sudo systemctl stop nginx'"  
        }  
    }  
}
```

```
post {  
    success {  
        echo "Blue-Green deployment successful."  
    }  
    failure {  
        echo "Deployment failed. Rolling back..."  
        sshagent(['deploy-key']) {  
            sh "ssh user@loadbalancer 'update_lb blue'"  
        }  
    }  
}
```

```
    }  
  }  
}  
}
```

GitLab CI/CD Example (Simplified)

yaml

stages:

- build
- deploy

build:

script:

- npm install
- npm run build

deploy-green:

stage: deploy

script:

- scp -r ./dist/* user@green_server:/var/www/html

- ssh user@loadbalancer './switch_green.sh'

Notes on Blue-Green Strategy

- Maintain two identical environments.
- Traffic is always live on one (e.g., Blue).
- Deploy to the other (e.g., Green), test it.
- Switch traffic using load balancer or DNS.
- If successful, Green becomes live; Blue can be used next time.